

Improvement of Planning Search Guidance via Active Instance Generation

Mingyu Hao¹, Sylvie Thiébaux^{1, 2} Felipe Trevizan¹

¹Australian National University, Australia

²LAAS-CNRS, Université de Toulouse, France

{mingyu.hao, felipe.trevizan}@anu.edu.au, sylvie.thiebaux@laas.fr

Abstract

State-of-the-art learning-for-planning systems utilise graph-based state encoding to derive feature embeddings for downstream policy and heuristic prediction. A major limitation is the model’s inability to generalise: features with few occurrences are poorly learned due to sparsity, and unseen features encountered during testing must currently be ignored. We address this feature generalisation gap by proposing a novel data-generation framework that synthesises new training samples guaranteed to contain targeted features. We show that this enables the effective deployment of active learning.

1 Introduction

The integration of statistical machine learning with graph-based state representations has enabled the derivation of heuristics and search guidance in classical planning (Shen, Trevizan, and Thiébaux 2020; Ferber, Helmert, and Hoffmann 2020; Chen, Trevizan, and Thiébaux 2024). Typically, these learning-for-planning pipelines rely on an offline supervised learning approach, mapping feature vectors derived from solved training instances to search guidance. However, this methodology encounters a generalisation bottleneck driven by two primary representational limitations.

The first limitation arises from the inherent sparsity of the training data. The resulting feature distribution is often highly skewed: a small number of dominant structural features appear in the majority of states, while rare features appear in only a fraction of the examples. This insufficient data coverage produces biased models that struggle to generalise across the broader feature space. The second limitation is specific to machine learning approaches that operate on a fixed-dimensional feature vector defined by the training set (Francès et al. 2019; Chen, Trevizan, and Thiébaux 2024; Hao et al. 2025). In these models the solver is forced to ignore out-of-distribution (OOD)/unseen features encountered in test instances. This structural omission removes relational context and reduces the heuristic’s capacity to extrapolate in complex, unseen planning instances.

To mitigate this feature generalisation gap, conventional systems employ online retraining, passively extracting states from test-instance search trajectories to augment the training set (Arfae, Zilles, and Holte 2010; Karia and Srivastava

2021). This passive retraining strategy exhibits three limitations. First, targeted OOD features may not appear along the solution trace, and even when they do, the sample size remains uncontrolled and often insufficient. Second, naive attempts to search the broader state space for missing features are computationally prohibitive and offer no guarantees of improvement. Finally, because complex test instances are computationally challenging to solve optimally, extracting ground-truth labels from sub-optimal search trajectories introduces label bias. This reinforces existing model errors and degrades the quality of learned search guidance.

In this paper, we propose a novel active-learning pipeline that resolves these limitations through active data synthesis. Rather than passively extracting data from biased test-set trajectories, the framework dynamically generates new planning instances explicitly guaranteed to contain the targeted rare or unseen features. When an OOD feature is encountered, it is translated into a set of structural constraints, expressed as PDDL axioms (Thiébaux, Hoffmann, and Nebel 2005). We then utilise the Answer Set Programming formulation by Grundke, Helmert, and Roger (2025) to synthesise small, solvable planning instances that satisfy these axioms and the domain constraints.

Crucially, this active generation mechanism grants us complete control over the training distribution: we can control the number of generated problems to secure an adequate sample size for the unseen features. Furthermore, these generated problems are structurally smaller and computationally simpler than the original test-set instances, allowing us to solve them optimally with a classical planner, thereby providing optimal labels for the retraining phase. Empirical evaluations across multiple domains demonstrate that our active-learning pipeline improves the overall problem-solving coverage of the learned search guidance.

2 Related Work

Existing methods for addressing the feature generalisation gap primarily focus on passive retraining. Bootstrap learning (Arfae, Zilles, and Holte 2010) and leapfrogging (Karia and Srivastava 2021) extract training data directly from test-instance search trajectories. However, this introduces label bias from sub-optimal plans and fails to guarantee adequate sampling of targeted OOD features. Marom and Rosman (2020) improves sample efficiency using uncertainty esti-

mates, but their backward-exploration task generation provides no explicit mechanism to isolate and synthesise instances containing specific OOD structures.

Alternatively, Pendurkar et al. (2024) organise training schedules via task-difficulty curricula. Because their task creation relies on random walks, it lacks control over feature distributions and cannot guarantee OOD feature acquisition. Our active synthesis approach complements this by generating samples with targeted OOD features on the fly.

Other works consider different task generation methodologies, for instance, Bettker et al. (2024) convert partial states obtained by goal regression into fully specified initial states using mutexes. However, classical planner-generated mutexes do not guarantee exhaustive constraint sets, often yielding invalid states that require resampling. In contrast, our framework enforces strict logical axioms to mathematically guarantee the validity of instances. A recent neuro-symbolic framework (Núñez-Molina, Mesejo, and Fernández-Olivares 2026) formulates problem generation as a Markov Decision Process and trains generative policies using Deep Reinforcement Learning to produce valid and diverse instances. While such methods generate computationally difficult benchmark distributions, they do not actively isolate and encode specific features into generated instances.

Finally, while recent policy learning work generates validation instances by scaling object counts to evaluate generalisation limits (Gros et al. 2025), these problems are not feature-driven and offer no mechanism for structural improvement. Our methodology directly extracts and translates OOD test features to actively synthesise targeted instances and explicitly resolve structural blind spots.

3 Background

3.1 Classical Planning

Let $|S|$ denote the size of a finite set S . A planning task of the kind we consider (Grundke, Helmert, and Roger 2025) is a tuple $\mathbf{P} = \langle \mathbf{D}, \mathbf{l} \rangle$ where $\mathbf{D}$ is the planning domain and $\mathbf{l}$ is the planning instance. The planning domain is a tuple $\mathbf{D} = \langle \mathcal{T}, \mathcal{C}, \mathcal{P}, \mathcal{A}, \mathcal{X} \rangle$ such that:

- $\mathcal{T}$ is a non-empty finite set of types. We assume a single-inheritance hierarchy over types (each type has exactly one parent except the root), with root type $\text{OBJECT} \in \mathcal{T}$. We define a partial order over $\mathcal{T}$ that represents the exact type hierarchy, denoted by $\sqsubseteq$. That is, $t_x \sqsubseteq t_y$ if t_y is the immediate parent of t_x and $t_x, t_y \in \mathcal{T}$. We write $t_x \sqsubseteq^* t_y$ if t_x is a descendant of t_y .
- $\mathcal{C}$ is a finite set of constant objects. Each object is assigned a leaf type $t \in \mathcal{T}$.
- $\mathcal{P}$ is a finite set of predicates which can be partitioned into basic predicates $\mathcal{P}_b$ and derived predicates $\mathcal{P}_d$, and $\mathcal{P}_b \cap \mathcal{P}_d = \emptyset$. Each predicate $p \in \mathcal{P}$ has the form $p(x_1 : t_1, \dots, x_k : t_k)$ where x_i is a parameter of type $t_i \in \mathcal{T}$, and $k \in \{0, \dots, a_{\max}\}$ is the arity of the predicate. $a_{\max}$ is the maximum arity of all basic predicates.
- $\mathcal{A}$ is a finite set of action schemas with preconditions and effects defined by a set of predicates in $\mathcal{P}_b$ whose arguments are parameters of the schema.

- $\mathcal{X}$ is a finite set of PDDL axioms (Thiébaux, Hoffmann, and Nebel 2005). A PDDL axiom is a rule of form $p_d(\bar{x}) \leftarrow l(\bar{x})$, where $p_d \in \mathcal{P}_d$ is a single derived predicate, and l is a first-order formula over predicates from $\mathcal{P}$, and $\bar{x}$ is a vector of free variables.

Definition 1 (Stratified Axiom Set). An axiom set $\mathcal{X}$ is stratified iff there exists a partition of the set of derived predicates $\mathcal{P}_d$ into non-empty subsets $\{\mathcal{P}_1, \dots, \mathcal{P}_m\}$ such that for each $p_i \in \mathcal{P}_i$ and axiom $(p_i(\bar{x}) \leftarrow l(\bar{x})) \in \mathcal{X}$:

- If predicate $p_j \in \mathcal{P}_d$ appears in the negation normal form of $l(\bar{x})$, then $p_j \in \mathcal{P}_j$ such that $j \leq i$.
- If predicate $p_j \in \mathcal{P}_d$ appears **negated** in the negation normal form of $l(\bar{x})$, then $p_j \in \mathcal{P}_j$ such that $j < i$.

The planning instance is a tuple $\mathbf{l} = \langle \mathcal{O}, s_0, \mathcal{G} \rangle$. $\mathcal{O}$ is a finite set of typed objects. An atom of form $p(o_1, \dots, o_k)$ is a ground instance of predicate $p(x_1 : t_1, \dots, x_k : t_k) \in \mathcal{P}$ where each parameter $x_i : t_i$ is substituted with an object $o_i \in \mathcal{O} \cup \mathcal{C}$ of type $t \sqsubseteq^* t_i$. Let $\mathcal{M}$ denote the set of all atoms, i.e. all possible ground instances of all predicates in $\mathcal{P}$. A **state** is a subset of $\mathcal{M}$ that contains only atoms grounded from $\mathcal{P}_b$. The state space $\mathcal{S}$ is the finite set of states of the planning task $\mathbf{P}$, $s_0 \in \mathcal{S}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{S}$ is a set of goal states.

An action a is a ground instance of an action schema from $\mathcal{A}$ and is applicable in a state s if and only if the precondition of a is a subset of s . We define a transition function f where $s' = f(s, a)$ means a is applicable in s and results in state s' . A plan π for $\mathbf{P}$ is a sequence of actions $a_1, \dots, a_n$ such that a_1 is applicable in s_0 and sequentially applying the actions leads to a goal state. More formally, there is a sequence of states $T_\pi = s_0, \dots, s_n$ such that $s_1 = f(s_0, a_1), \dots, s_n = f(s_{n-1}, a_n)$ and $s_n \in \mathcal{G}$. We call T_π a trace. An instance is **solved** if a plan is found. The optimal plan π^* of $\mathbf{P}$ is the plan with the least number of actions.

Given a state s from $\mathbf{P}$ with a stratified axiom set, we derive an extended state s_d . Starting with $s_d = s$ and the first stratum $\mathcal{P}_1$, we examine each derived predicate $p_d(\bar{x}) \in \mathcal{P}_1$. For each ground instance $p_d(\bar{o})$, where $\bar{o}$ is a vector of objects from $\mathcal{O} \cup \mathcal{C}$, if there is an axiom of form $p_d(\bar{x}) \leftarrow l(\bar{x})$ with every atom in $l(\bar{o})$ present in s_d then $p_d(\bar{o})$ is added to s_d . This process is then repeated for every stratum until the last one. A state s is valid if a special derived predicate $\text{legal}() \in s_d$ at the end of the process. A planning task is **valid** if its initial state is valid and there exists a valid state satisfying the goal condition.

3.2 Answer Set Program

There is a natural correspondence between domain axioms and an ASP program (Gebser et al. 2012).

Definition 2 (Answer Set Program). An answer set program (ASP) P , over a set of predicates $\mathcal{P}$ and a set of objects $\mathcal{C}$, is a finite set of rules. Each rule has the standard form

$$H(\bar{x}) \leftarrow \exists \bar{y} B(\bar{x}, \bar{y})$$

$H(\bar{x})$ is a disjunction of positive literals with a vector of free variables $\bar{x}$, and $B(\bar{x}, \bar{y})$ is a conjunction of positive and negative literals such that each variable of $\bar{x}$ is mentioned in

at least one positive literal. A rule is ground if every variable is substituted by a constant in $\mathcal{C}$. Let $H \leftarrow B$ denote a ground rule. We use B^+ and B^- to denote the conjunction of positive and negative predicates, respectively. A program is ground if all its rules are ground.

The **reduct** P^X of a ground program P relative to a set of atoms $X \subseteq \mathcal{M}$ is a program such that

$$P^X = \{H \leftarrow B^+ | X \models B^-, H \leftarrow B \in P\}$$

A set of atoms X is **closed** under a program P if $X \models B$ implies $X \models H$. A set of atoms X is a **stable model** of P if it is a subset-minimal set that is closed under P^X . In other words, a stable model is a smallest set of atoms that satisfies all the rules.

A rule of form $H \leftarrow$ with an empty body is a **fact** whose head must be satisfied in all stable models, and a rule of form $\leftarrow B$ with an empty head is an **integrity constraint** whose body must *not* be satisfied in any stable model. As syntactic sugar, we also consider rules of the form:

$$\{H(\bar{x}, \bar{y}) : C(\bar{y})\} = n \leftarrow$$

where $C(\bar{y})$ is a conjunction of positive literals and $n \geq 0$. This is a **cardinality constraint** requiring that stable models have exactly n instances $B(\bar{x}, \bar{y})$ with different values of $\bar{x}$, amongst those satisfying $C(\bar{y})$.

The connection between these programs and planning domains enables domain-independent instance generation. Recent work extends standard PDDL domain files with domain-wide constraints that formally define valid instances (Grundke, Röger, and Helmert 2024). This formalism was further extended to support object types, alongside a compilation method that translates these extended domains directly into ASP programs (Grundke, Helmert, and Roger 2025). Specifically, this compilation constructs ASP rules corresponding to the domain’s axioms and object type hierarchies. Then, given a predefined set of objects, an ASP solver such as Clingo (Gebser et al. 2019) can evaluate the program to generate valid domain instances. Each stable model of the resulting ASP program corresponds to a unique planning instance that is strictly guaranteed to satisfy all domain axioms. Moreover, for domains where deciding solvability is within polynomial complexity, additional domain axioms are used to ensure that valid tasks are also solvable.

We follow the axiom convention as in (Grundke, Röger, and Helmert 2024). More specifically, they define a valid instance via an axiom enforcing the legality predicate $legal()$:

$$legal() \leftarrow \neg illegal()$$

which Grundke, Helmert, and Roger (2025) translate into the ASP rules:

$$\begin{aligned} &\leftarrow \neg legal \\ legal &\leftarrow \neg illegal \end{aligned}$$

3.3 Weisfeiler-Leman Features

Given a planning task $\mathbf{P}$, a heuristic function $h : \mathcal{S} \rightarrow \mathbb{R}$ maps a state to a cost-to-goal estimate to guide search algorithms such as GBFS or A*. Many learning-based heuristics require a feature vector state representation (Shen, Trevizan,

and Thiébaux 2020; Karia and Srivastava 2021; Hao et al. 2025), with Weisfeiler-Leman (WL) features being one of the state-of-the-art methods for feature generation.¹

Computing the WL features of a planning task requires a graph representation of the task. We use the Typed Instance Learning Graph (TILG) (Bai, Thiébaux, and Trevizan 2025), a variant of the Instance Learning Graph (Chen, Trevizan, and Thiébaux 2024), which labels object nodes with their types and includes nodes representing static atoms.

Definition 3 (Typed Instance Learning Graph (TILG)). The TILG of a planning task $\mathbf{P} = \langle \mathbf{D}, \mathbf{l} \rangle$ is a graph $\mathbf{G} = \langle \mathbf{V}, \mathbf{E}, \mathbf{F}, \mathbf{L} \rangle$ where

- $\mathbf{V} = \mathcal{O} \cup \mathcal{C} \cup s_0 \cup \mathcal{G}$ is a set of nodes that represents an object or an atom true in the current state or the goal.
- $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{V}$ is a set of edges. They connect atom nodes to nodes representing their argument objects.
- $\mathbf{F} : \mathbf{V} \rightarrow \Sigma_{\mathbf{V}}$ where $\Sigma_{\mathbf{V}} = (\{ag, ug, aa\} \times \mathcal{P}) \cup \mathcal{T}$ mapping a node to a categorical node feature. A node representing an object is labelled by its type, while a node representing an atom is labelled according to whether this atom is true in both the goal and the current state (achieved goal, ag), only in the goal (unachieved goal, ug), or only in the current state (achieved atom, aa).
- $\mathbf{L} : \mathbf{E} \rightarrow \Sigma_{\mathbf{E}}$ where $\Sigma_{\mathbf{E}} = \{1, \dots, a_{max}\}$ mapping an edge to a categorical edge feature representing the object’s position in the argument list of the connected atom.

The neighbourhood of a node $u \in \mathbf{V}$ with respect to an edge label ι is defined as

$$\mathbf{N}_{\iota}(u) = \{v \in \mathbf{V} \mid e = \langle u, v \rangle \in \mathbf{E} \wedge \mathbf{L}(e) = \iota\}$$

To obtain the vector representation of the graph, the WL algorithm is applied iteratively to update each node’s feature. The algorithm requires an arbitrary injective hash function HASH and a parameter L specifying the number of iterations. Let $v \in \mathbf{V}$ be a node with initial feature $c^0(v) \in \Sigma_{\mathbf{V}}$. Writing $c^l(v)$ for v ’s feature at iteration l , the feature is updated by hashing a string that represents the node’s neighbourhood features and the corresponding edge labels:

$$c^{l+1}(v) \leftarrow \text{HASH}(c^l(v), \{(c^l(u), \iota) \mid \iota \in \Sigma_{\mathbf{E}}, u \in \mathbf{N}_{\iota}(v)\}) \quad (1)$$

Let $C_L = \cup_{l=0}^L \{c^l(v) \mid v \in \mathbf{V}\}$ be the set of unique features collected during a run of the WL algorithm. The WL feature representation of a TILG graph is a vector of length $|C_L|$, whose entries correspond to the number of times each feature was generated during a run of the WL algorithm. Example 1 demonstrates the computation process.

Example 1. Consider the Ferry example shown in Figure 1. There are two locations, l_1 and l_2 , and a ferry service carrying cars between them. Two cars c_1 and c_2 are currently at location l_1 . Car c_1 wants to go to location l_2 while car c_2 has arrived at its destination l_1 . For simplicity, we focus on the following basic predicates:

- $at_ferry(x : location)$: the ferry is at location x ;
- $at(x_1 : car, x_2 : location)$: car x_1 is at location x_2 .

¹There are other embedding methods, such as DLF (Martín and Geffner 2000), that can be explored in future work.

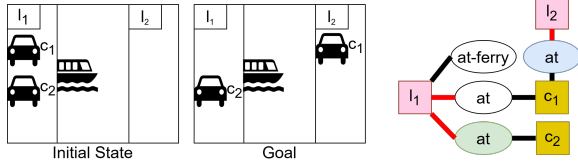


Figure 1: A Ferry instance with corresponding TILG.

There is no need to introduce actions. The initial state is: $\{at-ferry(l_1), at(c_1, l_1), at(c_2, l_1)\}$. And the goal is: $\{at(c_1, l_2), at(c_2, l_1)\}$. The corresponding TILG is shown on the right of Figure 1. In this graph, the colour and shape of each node $v \in \mathbf{V}$ indicate their initial features $c^0(v) \in \Sigma_V$. Square nodes represent objects. Pink nodes represent location-typed objects, yellow nodes represent car-typed objects. White circular nodes represent achieved atoms (aa), blue nodes represent unachieved goals (ug) and green nodes represent achieved goals (ag). The colour of the edges indicates the object's position as the argument. Let the edge feature function $\mathbf{L}$ map black edges to B and red edges to R . Then given a WLF generator, for the node v representing location a , at WL-iteration 1 the feature $c^1(v)$ is:

$$c^1(v) \leftarrow (location, \{(B, \langle at-ferry, aa \rangle), (R, \langle at, aa \rangle), (R, \langle at, ag \rangle)\})$$

The feature is calculated for each node and repeated L times to obtain $c^L(v)$ for every node $v \in \mathbf{V}$.

3.4 Learning WL Features For Search Guidance

The standard learning-for-planning pipeline follows an offline supervised training phase. A classical planner optimally solves training tasks $\mathcal{I}_{train}$ from a domain $\mathbf{D}$, e.g., via A* search guided by an admissible heuristic. A data extraction module then processes the resulting optimal plans and search trees to construct the training dataset. Learning targets depend on the chosen learning formulation. Examples include optimal distance-to-goal when learning a heuristic (Chen, Trevizan, and Thiébaux 2024), or relative state relationships when learning a state ranking (Hao et al. 2025).

Each extracted state is then embedded in a vector space using WL features by constructing a TILG for the planning task with the initial state replaced by the extracted state. This yields a fixed-dimensional feature vector for each state, where each dimension represents the frequency of a specific WL feature. An arbitrary learning model is trained on this data to induce a baseline search guidance function h . During testing, a search algorithm (e.g., Greedy Best-First Search) utilises h to solve unseen, larger and more complex tasks from a test set $\mathcal{I}_{test}$. As the solver encounters new states, it relies on the same WL generator to compute their feature representations.

A critical bottleneck emerges here: because the WL feature vector's dimensionality is fixed during offline training, the model is blind to any unseen features absent from $\mathcal{I}_{train}$. Consequently, the solver is forced to completely ignore these OOD features. This structural omission removes relational

Algorithm 1: Feature Reversion

Input: HASH_{rev} , list of features C_L , the feature to be reversed $c_{root} \in C_L$, the iteration $l \leq L$ of the feature

Output: partial TILG $\mathbf{G}$

```

1:  $\mathbf{G} = \langle \mathbf{V} = \{v_{root}\}, \mathbf{E} = \emptyset, \mathbf{F}, \mathbf{L} \rangle$ , s.t.  $c^l(v_{root}) = c_{root}$ 
2: while  $l > 0$  do
3:    $\text{NEWNODES} = \{\}$ 
4:    $\text{NEWEDGES} = \{\}$ 
5:   for  $v \in \mathbf{V}$  do
6:      $(c^{l-1}(v), \text{neighbours}) = \text{HASH}_{rev}(c^l(v))$ 
7:     for  $(c_i, \iota_i) \in \text{neighbours}$  do
8:        $u = \text{NODE}()$ 
9:        $c^{l-1}(u) = c_i$ 
10:       $e = \langle u, v \rangle$ 
11:       $\text{NEWNODES} = \text{NEWNODES} \cup \{u\}$ 
12:       $\text{NEWEDGES} = \text{NEWEDGES} \cup \{e\}$ 
13:       $\mathbf{L}(e) = \iota_i$ 
14:    end for
15:  end for
16:  for  $e = \langle u, v \rangle \in \mathbf{E}$  do
17:    select  $e' = \langle w, v \rangle \in \text{NEWEDGES}$ 
18:    s.t.  $\mathbf{L}(e') = \mathbf{L}(e) \wedge c^{l-1}(w) = c^{l-1}(u)$ 
19:     $\text{NEWNODES} = \text{NEWNODES} \setminus \{w\}$ 
20:     $\text{NEWEDGES} = \text{NEWEDGES} \setminus \{e'\}$ 
21:  end for
22:   $\mathbf{G} = \langle \mathbf{V} = \text{NEWNODES}, \mathbf{E} = \text{NEWEDGES}, \mathbf{F}, \mathbf{L} \rangle$ 
23:   $l = l - 1$ 
24: end while
25: for  $v \in \mathbf{V}$  do
26:    $\mathbf{F}(v) = c^0(v)$ 
27: end for
28: Return  $\mathbf{G}$ 

```

context, reducing the heuristic's capacity to guide the search effectively in unseen regions of the state space.

To remedy this, we develop a PDDL-axiom-based method for automatically generating valid, manageable instances that exhibit selected OOD features, and use it within a new pipeline for the active improvement of learned heuristics.

4 Instance Generation

Given a baseline planning task $\mathbf{P}$ and a targeted unseen feature c_{root} , the synthesis of new training samples proceeds through three phases:

1. **Feature Reversion:** Mapping the target WL feature back into its corresponding TILG subgraph, yielding partial initial state and goal.
2. **Task Synthesis:** Translating partial states into PDDL axiom constraints, and then generating a complete planning task via ASP.
3. **Data Extraction:** Solving the synthesised instance optimally to yield training targets.

4.1 Feature Reversion

The initial phase entails computing the preimage of the target feature under the WL Algorithm. This inversion relies

on the assumption that the hash function `HASH` used by `WL` is a perfect hash with respect to the observed feature space. In practice, this is trivially satisfied by mapping each feature to a unique integer.

The feature reversion procedure (Algorithm 1) starts with a partial TILG $\mathbf{G} = \langle \mathbf{V} = \{v_{root}\}, \mathbf{E} = \emptyset, \mathbf{F}, \mathbf{L} \rangle$, where the root node is assigned the target feature $c^l(v_{root}) = c_{root}$ at its corresponding iteration l . The algorithm reconstructs the graph $\mathbf{G}$ layer by layer, decrementing the iteration index l . At each step, for every node $v \in \mathbf{V}$, the inverse mapping `HASHrev` is applied to unpack the preceding feature $c^{l-1}(v)$ and its structural neighbourhood. The inverse mapping naturally follows the structure of Equation 1, with the base case mapping directly to the initial categorical features Σ_V :

$$\text{HASH}_{rev}(c^l(v)) = \begin{cases} (c^{l-1}(v), \{(c^{l-1}(u), \iota) \mid \iota \in \Sigma_E, u \in \mathbf{N}_\iota(v)\}) & \text{for } l > 0 \\ c^0(v) \in \Sigma_V & \text{for } l = 0 \end{cases}$$

For each unpacked neighbour feature and edge label (c_i, ι_i) , a provisional node u and edge $e = \langle u, v \rangle$ are generated and collected into temporary sets (`NEWNODES` and `NEWEDGES`). Because nodes from the previous iteration have already generated their neighbours, the algorithm performs a reconciliation step to prevent duplication. For any existing edge $e = \langle u, v \rangle \in \mathbf{E}$, it identifies and selects one corresponding provisional edge $e' = \langle w, v \rangle$ in `NEWEDGES` that shares the same edge feature ($\mathbf{L}(e') = \mathbf{L}(e)$) and destination feature ($c^{l-1}(u) = c^{l-1}(w)$). For each edge, only one provisional node should be identified as a duplicate. Then redundant provisional nodes (w) and edges (e') are discarded. The remaining new nodes and edges are then added to the partial TILG $\mathbf{G}$. This process terminates when the base iterations conclude, at which point all $v \in \mathbf{V}$ are assigned the base categorical features $c^0(v)$.

It is crucial to note that while cycles may exist in the original TILG, the recovered graph $\mathbf{G}$ is strictly acyclic (forming a tree), with the node v_{root} as the root node. This is because the `WL` algorithm cannot distinguish between cyclic structures and their isomorphic tree unfoldings based solely on the hash of a single node.

Based on the node categorical features, the vertices of $\mathbf{G}$ are partitioned into object nodes $\mathbf{V}_O = \{v \in \mathbf{V} \mid \mathbf{F}(v) \in \mathcal{T}\}$ and predicate nodes $\mathbf{V}_P = \{v \in \mathbf{V} \mid \mathbf{F}(v) \in \{ag, ug, aa\} \times \mathcal{P}\}$. Each predicate node connects to object nodes corresponding to its arguments, with the edge feature $\iota \in \Sigma_E$ denoting the positional index. Due to the bounded nature of the `WL` iterations, frontier nodes in the recovered graph have incomplete neighbourhood structures. As a result, if a frontier node represents a predicate, only one of its arguments (the one whose node is expanded in the last iteration) is explicitly instantiated, leading to a partial TILG. This partial TILG naturally decomposes into a partial initial state s_i (comprising atoms labelled *aa* or *ag*) and a goal s_g (atoms labelled *ag* or *ug*). We write O_G for the set of all unique objects within this recovered TILG.

4.2 Task Synthesis

This phase reconstructs a fully specified, domain-compliant planning instance that entails the derived partial state s_i and

goal s_g . Formulating s_i and s_g as integrity constraints (axioms) within an extended planning domain guarantees that any valid instance generated from this domain inherently exhibits the targeted unseen features.

Formally, we define an extended planning domain $\mathbf{D}_{gen} = \langle \mathcal{T}, \mathcal{C}_{gen}, \mathcal{P}, \mathcal{A}, \mathcal{X}_{gen} \rangle$, which inherits the dynamics of the baseline domain $\mathbf{D}$. The constant set is augmented such that $\mathcal{C}_{gen} = \mathcal{C} \cup O_G$, effectively fixing the recovered objects O_G as domain constants.

For each k -ary predicate node in the partial TILG, the translation into a domain axiom breaks down into three distinct cases, contingent upon whether this node is fully expanded in the TILG. The first case handles fully grounded atoms where all argument objects $o_1, \dots, o_k \in \mathcal{C}_{gen}$. The second case addresses partially grounded atoms (frontier nodes) where only a single argument o_i is in $\mathcal{C}_{gen}$ for $i \in \{1, \dots, k\}$, while all other arguments are not.

In the first case, the fully specified atom maps directly to an ASP fact:

Case 1. Given the extended domain $\mathbf{D}_{gen}$ and an atom of form $p(o_1, \dots, o_k)$ with $o_1, \dots, o_k \in \mathcal{C}$, we add the following axioms to the domain:

$$\text{illegal}() \leftarrow \neg p(o_1, \dots, o_k)$$

Which is translated into the following ASP rule:

$$p(o_1, \dots, o_k) \leftarrow$$

In the second case, the partially specified atom mandates an existential constraint over the unknown arguments:

Case 2. Given the extended domain $\mathbf{D}_{gen}$ and an atom of the form $p(x_1, \dots, x_k)$ with the i -th parameter $x_i = o \in \mathcal{C}_{gen}$, $i \in \{1, \dots, k\}$, we add the following axiom into the domain:

$$\text{illegal}() \leftarrow \neg \exists \bar{x} p(\bar{x}) \wedge \bar{x}_i = o$$

Which is translated into ASP rules by introducing an intermediate atom $\text{atom}_{p,i}$:

$$\begin{aligned} \text{illegal} &\leftarrow \neg \text{atom}_{p,i}(o) \\ \text{atom}_{p,i}(o) &\leftarrow \exists \bar{x} p(\bar{x}) \wedge \bar{x}_i = o \end{aligned}$$

A problem arises when multiple partially specified atoms share the same predicate p and an identical i -th object argument. The standard existential axiom is insufficient, as it does not enforce the distinctness of these atoms. To resolve this, we introduce an exact-count quantifier $\exists^n$, enforcing the existence of exactly n unique ground instances.

Case 3. Given an extended domain $\mathbf{D}_{gen}$ and n atoms of the form $p(x_1, \dots, x_k)$ that all share the same predicate p and the same i -th argument $o_i = o \in \mathcal{C}_{gen}$, we add the following axiom to the domain:

$$\text{illegal}() \leftarrow \neg \exists^n \bar{x} p(\bar{x}) \wedge \bar{x}_i = o$$

Which is translated into the following ASP rule:

$$\{p(\bar{x}) : \bar{x}_i = o\} = n \leftarrow$$

The final step employs an ASP-based instance generator (Grundke, Helmert, and Roger 2025) to synthesise a valid task for the extended domain $\mathbf{D}_{gen}$. The generator compiles

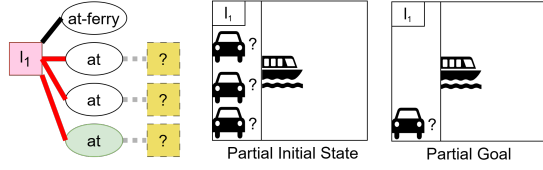


Figure 2: Example of Partial TILG from Feature Reversion and corresponding partial states and goal.

the domain dynamics and the newly injected axioms into an ASP logic program and solves it to yield a valid planning instance. Example 2 gives an example of the conversion from a feature to ASP rules.

Once synthesised, these generated tasks are optimally solved using a classical planner, processed by the WL feature generator to extract structural encodings, and finally supplied to the learner to update the search guidance function.

Example 2. Assume that by collecting features from a Ferry instance, we have obtained an iteration-1 feature c^1 and that by applying the HASH_{rev} function, it reveals:

$$\begin{aligned} c^0 &= \text{HASH}_{rev}(c^1) \\ &= (\text{location}, \{ (B, \langle \text{at-ferry}, aa \rangle), (R, \langle \text{at}, aa \rangle), \\ &\quad (R, \langle \text{at}, aa \rangle), (R, \langle \text{at}, ag \rangle) \}) \end{aligned}$$

The text corresponds to the partial TILG illustrated on the left of Figure 2. Nodes and edges with a dashed outline represent the parameters of predicates that cannot be expanded from the feature but can be derived from the definitions of the predicates. The set of constant objects $O_c = \{l_1 : \text{location}\}$. The result of the partial initial state is:

$$\{\text{at-ferry}(l_1), \text{at}(?, l_1), \text{at}(?, l_1), \text{at}(?, l_1)\}$$

The partial state indicates the ferry and three cars are at location l_1 . The partial goal is $\{\text{at}(?, l_1)\}$, indicating one of the three cars has arrived at its destination. The corresponding domain axioms for the initial state are:

$$\begin{aligned} \text{illegal}() &\leftarrow \neg \text{at-ferry}(l_1) \\ \text{illegal}() &\leftarrow \neg \exists^2(x_1 : \text{car}, x_2 : \text{location}) \\ &\quad \text{at}(x_1, x_2) \wedge x_2 = l_1 \end{aligned}$$

The corresponding ASP rules are:

$$\begin{aligned} \text{at-ferry}(l_1) &\leftarrow \\ \{\text{at}(x_1 : \text{car}, x_2 : \text{location}) : x_2 = l_1\} = 2 &\leftarrow \end{aligned}$$

The axiom for the goal is:

$$\text{illegal}() \leftarrow \neg \exists(x_1 : \text{car}, x_2 : \text{location}) \text{at}(x_1, x_2) \wedge x_2 = l_1$$

and the ASP rule is:

$$\begin{aligned} \text{illegal} &\leftarrow \neg \text{atom}_{\text{at},2}(l_1) \\ \text{atom}_{\text{at},2}(o : \text{location}) &\leftarrow \exists(x_1 : \text{car}, x_2 : \text{location}) \\ &\quad \text{at}(x_1, x_2) \wedge x_2 = o \end{aligned}$$

5 The Active-Learning Pipeline

Having established the mechanics of feature reversion and axiom-based task generation, we now formalise the complete active-learning pipeline. To ensure broad applicability across different learning architectures, the pipeline is modular, with the generalised procedure detailed in Algorithm 2. The algorithm requires a testing set $\mathcal{I}_{test}$ and a baseline search guidance function h . It utilises three configurable modules: FEATSEL to select a subset of unseen features, DATAGEN to synthesise and extract new data containing these features, and LEARN to translate the extracted data into an updated search guidance function.

To manage unseen features without disrupting established knowledge, the system decouples them from the baseline model by fixing h and initialising a new guidance function h_{re} . While h maps features observed during offline learning to search guidance, h_{re} is dedicated exclusively to evaluating unseen features. This decoupling is designed to address the imbalance of data between training and testing, since the testing problems are much larger than the training problems. Early experiments confirmed that maintaining the fixed feature weights from the training set results in better overall learning outcomes. An aggregation function, COMBINE, merges these outputs to produce a unified state evaluation, effectively making h_{re} learn to fix the prediction of h based on unseen features. When both h and h_{re} return numerical outputs, COMBINE is defined as the sum of their evaluations.

As the solver iterates through each test instance $i \in \mathcal{I}_{test}$, it runs Greedy Best-First Search (GBFS) guided by the combined search guidance. It extracts the solution plan π_i , the solution trace T_i , and a set of unseen features C_i encountered during the search. If the set of unseen features is not empty ($C_i \neq \emptyset$), it triggers the update of h_{re} . The FEATSEL module filters C_i to identify the targeted features. We use the top- k most frequently occurring unseen features. In future work, it is worth exploring different unseen feature selection strategies. The DATAGEN module takes the selected features and module-specific arguments $\{args\}$ to synthesise training data that encompasses these features. This new data is appended to the cumulative *Dataset*. Finally, the LEARN module processes this newly generated data to update the function h_{re} . The solver then proceeds to the next test instance using the updated function.

6 Experiments

We evaluate our framework on the benchmark domains from the IPC 2023 Learning Track (IPC23LT) (Segovia and Seipp 2023). Each domain contains 99 training and 90 testing tasks with increasing difficulty. For the synthesis of new tasks, we utilise the domain axioms formulated by (Grundke, Röger, and Helmert 2024). These axioms cover all IPC23LT domains except Sokoban because deciding its solvability is PSPACE-complete (Culberson 1997), and the employed ASP formalism captures only complexity class P. We relaxed some axioms to allow a broader range of instances to serve as valid samples while preserving guaranteed solvability. For instance, in Spanner, the original axioms forced

Algorithm 2: Retraining Algorithm

Input: $\mathcal{I}_{test}, h$
Module: FEATSEL, DATAGEN, LEARN

```

1:  $h_{re} = 0$ 
2:  $Dataset = \emptyset$ 
3: for  $i \in \mathcal{I}_{test}$  do
4:    $\pi_i, T_i, C_i = \text{GBFS}(i, \text{COMBINE}(h, h_{re}))$ 
5:   if  $C_i \neq \emptyset$  then
6:      $C_{sel} = \text{FEATSEL}(C_i)$ 
7:      $Dataset = Dataset \cup \text{DATAGEN}(C_{sel}, \{args\})$ 
8:      $h_{re} = \text{LEARN}(Dataset)$ 
9:   end if
10: end for

```

the person to begin at the Shed in the initial state, whereas in our generated instances, they may need to begin at arbitrary locations. All experiments are conducted on a 64-core AMD Ryzen 5995WX CPU with 32GB of memory.

The training set instances are solved using A* search with the LM-cut Heuristic (Helmert and Domshlak 2009), except for Blocksworld, which is solved using the Potential Heuristic (Seipp, Pommerening, and Helmert 2015). The Potential Heuristic is utilised exclusively for the Blocksworld because empirical tests indicated that when LM-cut is used, only a single retraining is triggered throughout the learning process. As a result, it cannot demonstrate the difference between retraining and no retraining. We learn both the initial and retrained search guidance functions, h and h_{re} , using the Search-Space Ranking-based Linear Program model (Chen, Trevizan, and Thiébaux 2024; Hao et al. 2025). For each solved instance in the training set, we extract Search-Space ranking pairs from the A* search trees and learn a pairwise-ranking function, and then convert it into a pointwise ranking function as the baseline guidance function h . We follow the testing procedure of (Hao et al. 2025) and use GBFS with the learned function to solve each testing instance with a 30-minute cutoff. We select the top five most frequent unseen features and experiment with four data generators (DATAGEN):

NONE: as a baseline, the initial model is evaluated on the test set without retraining ($\text{DATAGEN} = \emptyset$).

PLAN: as a naive approach, when retraining is triggered, it uses states with unseen features from the solution trace. The data generator requires the solution plan and trace as additional parameters ($\{args\} = \{\pi_i, T_i\}$). Since this setting requires a plan for data generation, if an instance is not solved, we cannot retrain even if we have recorded unseen features during the search. Moreover, the solution trace may not contain the unseen features found during the search, in which case the training also has to be skipped. Finally, the solution is not guaranteed to be optimal. Longer plans quickly exhaust the memory budget when extracting data using Search-Space ranking. Therefore, for retraining in this setting, we restrict the Search-Space Ranking to the states within 1 step from the solution, as in (Hao et al. 2024).

AXIOM: this setting uses the proposed instance-

Method	BW	CS	FR	FL	MI	RO	SA	SP	TP	Σ
NONE	72	53	74	38	72	33	40	67	34	483
PLAN	81	43	74	38	73	35	40	40	35	459
AXIOM	76	48	81	38	80	36	44	73	36	512
COMBINE	81	50	81	38	79	37	44	40	36	486

Table 1: Coverage of the various retraining methods. The best coverage in each column is in bold.

generation method to generate valid instances for the selected unseen features. The generator requires two additional parameters: n representing the number of tasks to be synthesised per unseen feature, and t representing the timeout per task generation ($\{args\} = \{n, t\}$). We use a 30-minute timeout and we generate only a single instance per unseen feature. The generated tasks are solved using A* with the LM-cut heuristic with a 10-minute timeout. Since the generated tasks are generally much smaller than those in the testing set, new training data are extracted from the solved tasks using Search-Space Ranking without restricting states’ distance from the solution.

COMBO: this setting combines both the naive and the axiom-based method and re-trains with both the states from the solution and the generated instances. Therefore, the data generator requires parameters from both methods ($\{args\} = \{\pi_i, T_i, n, t\}$). The data generation procedures follow exactly those outlined above for each method, and the datasets are merged at the end.

7 Results

For all our results, we abbreviate the domains as follows: blockworld (BW), childsnack (CS), ferry (FR), floortile (FL), miconic (MI), rovers (RO), satellite (SA), spanner (SP) and transport (TP). Table 1 summarises the coverage for the baseline and the three retraining strategies.

The naive approach achieved a total coverage of 459 instances, a decrease of 4.96% from the baseline. The improvement in coverage across BW, MI, RO, and TP was offset by worse performance in CS and SP. Our axiom-based approach achieved the highest overall coverage, providing a 6% increase over the baseline. The coverage increases are particularly obvious in domains such as FR, MI, and SP. This demonstrates that generating targeted, optimally solvable instances for OOD features can expand the model’s representation space without degrading baseline performance. The COMBO approach yielded a total coverage 0.6% higher than the baseline and 5% lower than the AXIOM approach. While it slightly exceeds the baseline, the coverage variance across domains is high. For domains such as CS and RO, the COMBO approach shows an improvement. However, in other domains such as SP and MI, it achieved lower coverage than the AXIOM approach. This indicates that integrating biased data from suboptimal trajectories offsets the performance gains from optimally generated states.

What happened to Spanner? The most significant coverage drop for the PLAN and COMBO methods occurs in SP,

where both solve 27 fewer instances than the baseline. In this domain, an agent must collect spanners along a one-way path before arriving at a terminal to tighten a set of nuts. Navigating to the gate without enough spanners results in a dead end. These methods fail because they cannot generalise to prioritise picking up more spanners on harder tasks. Because they extract training data exclusively from short, suboptimal solution traces of easier tasks, the models learn a biased upper bound on spanners (capping at approximately 26 spanners). When evaluated on medium-difficulty tasks requiring a larger inventory, the retrained heuristics confidently, yet incorrectly, guide the agent to the gate without picking up enough spanners. Upon reaching the gate, the domain’s dead-end property traps the solver, who exhausts the time limit exploring permutations of the limited spanners and remaining nuts. In contrast, the AXIOM method generalises successfully. By training on generated optimal tasks, it learns to collect excess spanners early in the path. Although this results in sub-optimal plans, the strategy produces robust trajectories that prevent dead ends.

How do the new features help? We performed an empirical analysis of the distribution of unseen features. It indicates that the majority of frequently appearing unseen features do not represent entirely new relational concepts. Instead, they represent scalar increases in existing topological structures. For instance, in the Ferry domain, a location with three cars yields a distinct feature from one with four cars.

To isolate the framework’s ability to adapt to these count-based structural shifts, we constructed a constrained variant of the domain, DUMMY-FERRY. This domain contains exactly two locations (a and b), requiring the solver to transport a number of cars from a to b . The optimal generalised solution is a repetitive sequence: load a car, sail to b , unload, and return to a . Figure 3 illustrates the solving time across 60 tasks of increasing car counts (from 1 car to 60 cars). The baseline model (NONE), trained with 10 instances with 1 to 10 cars, solved only 21 of the 60 test instances. As the number of cars scales, the baseline solving time grows exponentially, quickly exceeding the time limits. The naive approach (PLAN) performs only marginally better. It solves 22 instances, but exhibits search instability before experiencing the exact same exponential timeout failure. This confirms that extracting data from sub-optimal solution traces is insufficient for learning a generalisable counting policy.

Conversely, the AXIOM approach successfully solved 44 test tasks. By generating targeted tasks to learn these unseen count-based features, the active-learning pipeline enabled the heuristic to evaluate the scaled state spaces. The solver cannot progress further because the classical planner fails to solve the generated tasks, leading to skipped retraining. Consequently, the model no longer learns new features.

8 Conclusion

This paper addressed the feature generalisation bottleneck in learning-for-planning systems, where fixed-dimensional embeddings fail to evaluate OOD features during deployment. Empirical experiments showed that online retraining

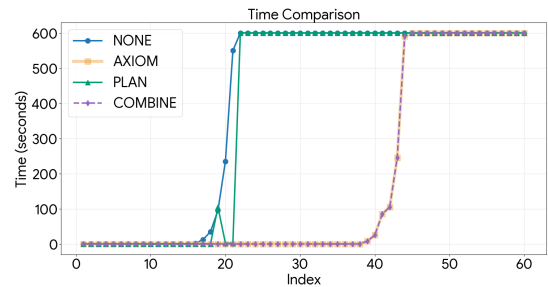


Figure 3: Solving time of the Dummy-Ferry tasks.

on sub-optimal test trajectories introduces label bias and degrades search performance. To resolve this, we introduced an active-learning pipeline that utilises ASP to generate optimally solvable tasks guaranteed to contain targeted features. Empirical evaluations confirmed that this active generation framework eliminates label bias and significantly improves overall problem-solving coverage.

Future work will explore three primary extensions. First, developing adaptive feature-selection strategies, such as prioritising features based on predictive uncertainty, to further optimise learning efficiency. Second, integrating this active synthesis loop with curriculum learning frameworks (Pendurkar et al. 2024) to provide tailored instances based on feature distribution rather than difficulty. Finally, we aim to adapt this synthesis methodology for alternative representation architectures, such as Graph Neural Networks, and expand the ASP compilation logic to support a broader set of complex PDDL domain dynamics.

References

- Arfaee, S. J.; Zilles, S.; and Holte, R. 2010. Bootstrap Learning of Heuristic Functions. In *Proceedings of the 3rd International Symposium on Combinatorial Search (SoCS)*, 11, 52–60.
- Bai, Y.; Thiébaux, S.; and Trevizan, F. 2025. Learning efficiency meets symmetry breaking. In *Proceedings of the 35th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 35, 154–159.
- Bettker, R. V.; Minini, P. P.; Pereira, A. G.; and Ritt, M. 2024. Understanding sample generation strategies for learning heuristic functions in classical planning. *Journal of Artificial Intelligence Research (JAIR)*, 80: 243–271.
- Chen, D. Z.; Trevizan, F.; and Thiébaux, S. 2024. Return to tradition: learning reliable heuristics with classical machine learning. In *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 34, 68–76.
- Culberson, J. 1997. Sokoban is PSPACE-complete. *Technical Report TR 97-02, Department of Computing Science, The University of Alberta, Edmonton, Alberta, Canada*.
- Ferber, P.; Helmert, M.; and Hoffmann, J. 2020. Neural network heuristics for classical planning: A study of hyperparameter space. In *Proceedings of the 24th European Con-*

- ference on Artificial Intelligence (ECAI), 2346–2353. IOS Press.
- Francès, G.; Corrêa, A. B.; Geissmann, C.; and Pommerening, F. 2019. Generalized Potential Heuristics for Classical Planning. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 5554–5561. Macao, China. ISBN 978-0-9992411-4-1.
- Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2012. *Answer Set Solving in Practice*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers.
- Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1): 27–82.
- Gros, T. P.; Müller, N. J.; Fišer, D.; Valera, I.; Wolf, V.; and Hoffmann, J. 2025. Per-domain generalizing policies: On validation instances and scaling behavior. In *Proceedings of the 35th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 35, 198–203.
- Grundke, C.; Helmert, M.; and Roger, G. 2025. Domain-Independent Instance Generation for Classical Planning. In *Proceedings of the 22nd International Conference on Principles of Knowledge Representation and Reasoning (KR)*.
- Grundke, C.; Röger, G.; and Helmert, M. 2024. Formal representations of classical planning domains. In *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 34, 239–248.
- Hao, M.; Chen, D. Z.; Trevizan, F.; and Thiébaux, S. 2025. Effective Data Generation and Feature Selection in Learning for Planning. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI)*.
- Hao, M.; Trevizan, F.; Thiébaux, S.; Ferber, P.; and Hoffmann, J. 2024. Guiding GBFS through learned pairwise rankings. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 6724–6732. International Joint Conferences on Artificial Intelligence Organization.
- Helmert, M.; and Domshlak, C. 2009. Landmarks, Critical Paths and Abstractions: What’s the Difference Anyway? In *Proceedings of the 10th International Conference on Automated Planning and Scheduling (ICAPS)*.
- Karia, R.; and Srivastava, S. 2021. Learning Generalized Relational Heuristic Networks for Model-Agnostic Planning. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI)*.
- Marom, O.; and Rosman, B. 2020. Utilising Uncertainty for Efficient Learning of Likely-Admissible Heuristics. In *Proceedings of the 30th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 30, 560–568.
- Martín, M.; and Geffner, H. 2000. Learning Generalized Policies in Planning Using Concept Languages. In *Proceedings of the 7th International Conference on Principles of Knowledge Representation and Reasoning (KR)*.
- Núñez-Molina, C.; Mesejo, P.; and Fernández-Olivares, J. 2026. Automated planning instance generation with neuro-symbolic AI. *Artificial Intelligence*, 352: 104471.
- Pendurkar, S.; Lelis, L. H.; Sturtevant, N. R.; and Sharon, G. 2024. Curriculum Generation for Learning Guiding Functions in State-Space Search Algorithms. In *Proceedings of the 17th International Symposium on Combinatorial Search (SoCS)*, volume 17, 91–99.
- Segovia, J.; and Seipp, J. 2023. Benchmarking Repository of IPC 2023 - Learning Track. <https://github.com/ipc2023-learning/benchmarks>.
- Seipp, J.; Pommerening, F.; and Helmert, M. 2015. New optimization functions for potential heuristics. In *Proceedings of the 25th International Conference on Automated Planning and Scheduling (ICAPS)*, volume 25, 193–201.
- Shen, W.; Trevizan, F.; and Thiébaux, S. 2020. Learning Domain-Independent Planning Heuristics with Hypergraph Networks. In *Proceedings of the 30th International Conference on Automated Planning and Scheduling (ICAPS)*.
- Thiébaux, S.; Hoffmann, J.; and Nebel, B. 2005. In defense of PDDL axioms. *Artificial Intelligence*, 168(1-2): 38–69.